# 第2次練習題目-隨堂-QZ2

> 學號：113111130 (換成自己的)
> 姓名：黃貫栢 (換成自己的)
> 作業撰寫時間：. (mins，包含程式撰寫時間，換成自己的)
> 最後撰寫文件日期：2026/10/28 (換成自己的)

本份文件包含以下主題：(至少需下面兩項，若是有多者可以自行新增)

- ☑ 說明內容
- ☑ 其他 (可以包含心得或是想跟老師反映)

## 說明內容

開始寫說明，該說明需說明想法， 並於之後再對上述想法的每一部分將程式進一步進行展現， 若需引用程式區則使用下面方法， 若為.cs檔內程式除了於敘述中需註明檔案名稱外， 還需使用語法```語言種類 程式碼 ```，其中語言種類若是要用python則使用py，java則使用java，C/C++則使用cpp， 下段程式碼為語言種類選擇csharp使用後結果：

```
public void mt_getResult(){
    ...
}
```

若要於內文中標示部分網頁檔，則使用以下標籤```html 程式碼 ```， 下段程式碼則為使用後結果：

```
<%@ Page Language="C#" AutoEventWireup="true" ...>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<meta http-equiv="Content-Type" ...>
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
        </div>
    </form>
</body>
</html>
```

更多markdown方法可參閱https://ithelp.ithome.com.tw/articles/10203758

請在撰寫"說明程式與內容"該塊內容，請把原該塊內上述敘述刪除，該塊上述內容只是用來指引該怎麼撰寫內容。

# 1. (需截圖說明，並且需截圖執行結果)

Ans:

選項 PlayerController.cs

Assembly-CSharp | PlayerController | Update()

```
using System.Collections;                   // 引入 C# 基本集合命名空間（提供非泛型集合與協程支援）
using System.Collections.Generic;           // 引入 C# 泛型集合命名空間（提供 List、Dictionary 等型別）
using UnityEngine;                          // 引入 Unity 核心引擎命名空間（提供 MonoBehaviour、Debug 等核心功能）
using UnityEngine.InputSystem;              // 引入 Unity 新版輸入系統命名空間（提供 Keyboard 等輸入 API）

0 個參考
public class PlayerController : MonoBehaviour // 宣告名為 PlayerController 的公開類別，繼承 MonoBehaviour 即可掛載至遊戲物件
{                                           // 類別本體開始
                                            // 預設註解：Start 會在第一個影格更新前被呼叫
    0 個參考
    void Start()                            // 生命週期函式：物件啟用時只會執行一次，用於初始化
    {                                       // Start 函式區塊開始

    }                                       // Start 函式區塊結束

                                            // 預設註解：Update 每一影格都會被呼叫一次
    0 個參考
    void Update()                           // 生命週期函式：每一影格更新時自動執行，用於即時輸入偵測與更新邏輯
    {                                       // Update 函式區塊開始
        // 取得現在鍵盤的狀態
        Keyboard keyboard = Keyboard.current; // 宣告 Keyboard 型別變數，取得目前連接並作用中的鍵盤裝置實例

        // 偵測玩家有沒有按 W 鍵
        if (keyboard.wKey.isPressed)        // 條件判斷：當鍵盤上的 W 鍵處於「被按住」狀態時回傳 true
        {                                   // if 判斷區塊開始
            Debug.Log("玩家按了 W 鍵！");   // 在 Unity 編輯器的主控台 (Console) 輸出指定的文字訊息
        }                                   // if 判斷區塊結束
    }                                       // Update 函式區塊結束

}                                           // 類別本體結
```




# 2. (需截圖說明，並且需截圖執行結果)

Ans:

```
    void Update()                           // 生命週期函式：每一影格更新時自動執行，用於即時輸入偵測與更新邏輯
    {                                       // Update 函式區塊開始
        // 取得現在鍵盤的狀態
        Keyboard keyboard = Keyboard.current; // 宣告 Keyboard 型別變數，取得目前連接並作用中的鍵盤裝置實例

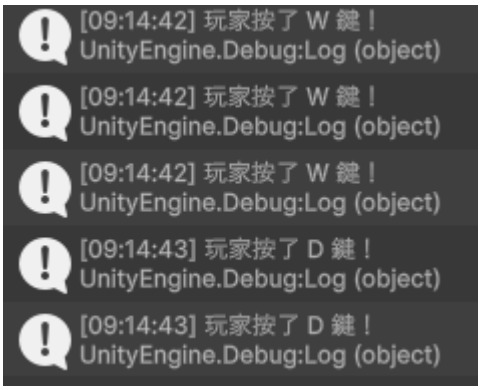
        // 偵測玩家有沒有按 W 鍵
        if (keyboard.wKey.isPressed)        // 條件判斷：當鍵盤上的 W 鍵處於按住狀態時
        {                                   // W 鍵 if 判斷區塊開始
            Debug.Log("玩家按了 W 鍵！");   // 在 Console 印出 W 鍵訊息
        }                                   // W 鍵 if 判斷區塊結束

        // 偵測玩家有沒有按 A 鍵
        if (keyboard.aKey.isPressed)        // 條件判斷：使用 aKey 偵測 A 鍵是否處於按住狀態
        {                                   // A 鍵 if 判斷區塊開始
            Debug.Log("玩家按了 A 鍵！");   // 在 Console 印出 A 鍵訊息
        }                                   // A 鍵 if 判斷區塊結束

        // 偵測玩家有沒有按 S 鍵
        if (keyboard.sKey.isPressed)        // 條件判斷：使用 sKey 偵測 S 鍵是否處於按住狀態
        {                                   // S 鍵 if 判斷區塊開始
            Debug.Log("玩家按了 S 鍵！");   // 在 Console 印出 S 鍵訊息
        }                                   // S 鍵 if 判斷區塊結束

        // 偵測玩家有沒有按 D 鍵
        if (keyboard.dKey.isPressed)        // 條件判斷：使用 dKey 偵測 D 鍵是否處於按住狀態
        {                                   // D 鍵 if 判斷區塊開始
            Debug.Log("玩家按了 D 鍵！");   // 在 Console 印出 D 鍵訊息
        }                                   // D 鍵 if 判斷區塊結束
    }                                       // Update 函式區塊結束

}                                           // 類別本體結束
```







# 其他